

Introduce yourself in the chat!

Tell us your program and concentration

Hello there



I'm Omer (the batter on the left), your instructor for today!

- Studied Management Science and Computer Science in my undergrad
- Graduated from the MI program at the iSchool where I studied Data Science
- Working at the Ontario Public Service as a Data Scientist
- And I love teaching

Housekeeping

- This session is interactive! Please ask questions when prompted, I'll be stopping regularly to check if everyone is understanding the concepts
- We will have a short break in the middle (stretch, grab some tea)
- In case, we aren't able to cover something properly, we will skip it. The goal is to learn the content, not rush through
- We will be doing some coding together
 - Sometimes you'll code and we then go through the solution together!
- You will have access to the slides throughout in case you need to go back to a particular slide
- I'll be giving you some homework before the next workshop

Here's What We'll Cover

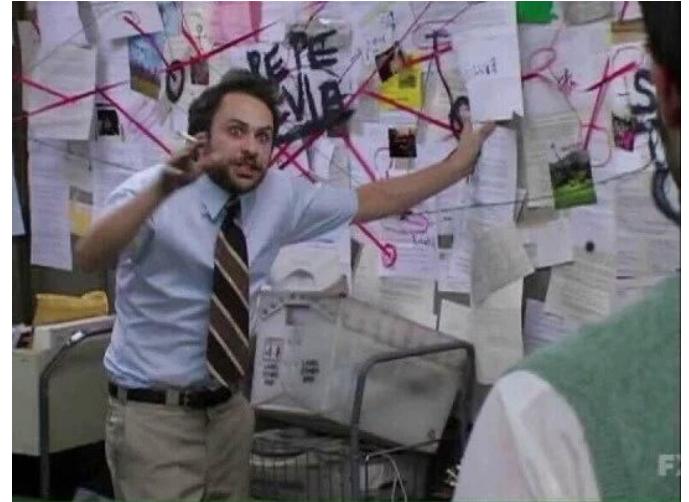
1. Importing data into Python
2. Cleaning and Transforming your dataset
3. Saving your datasets

Prologue

Why are we studying this anyway

Who doesn't love a nice story?

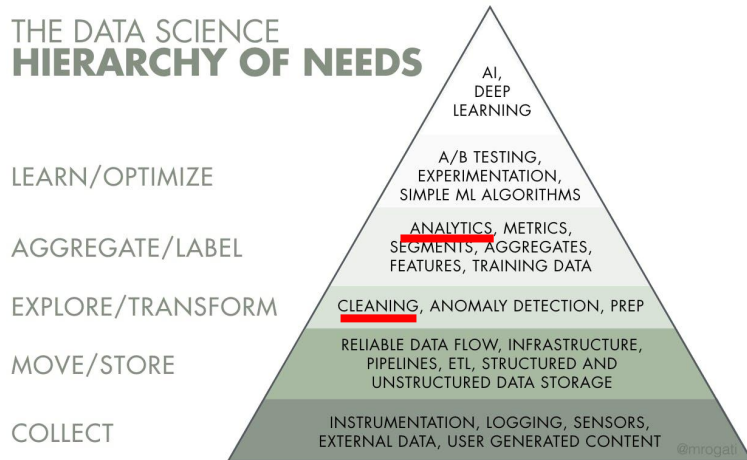
- Can anyone recall an engaging story?
 - Why'd you like it?
 - What are some important components of a story?
- Most, if not all data analysis, is trying to tell a story too!
- Here's what you'll need according Harvard Business School:
 - Data
 - Narrative
 - Visualizations



Don't tell a story like this

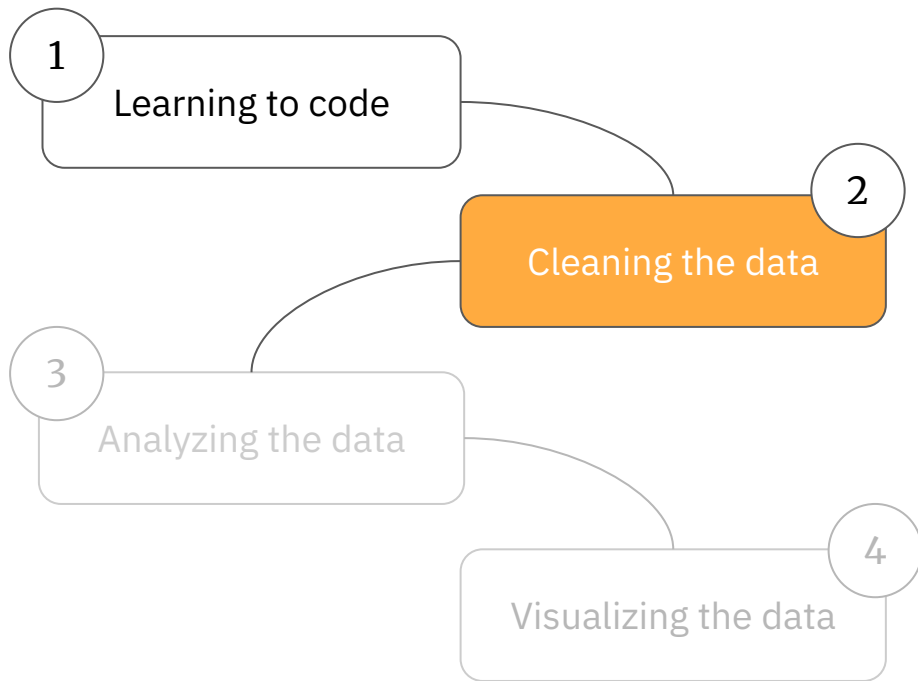
The data field is booming!

- What I will be showing you today is a little glimpse of data analytics
- There are so many adjacent fields that are highly valued in all industries
- What's the secret sauce?
 - **Domain Expertise**
 - And of course, **practice**



[Source](#)

Our journey



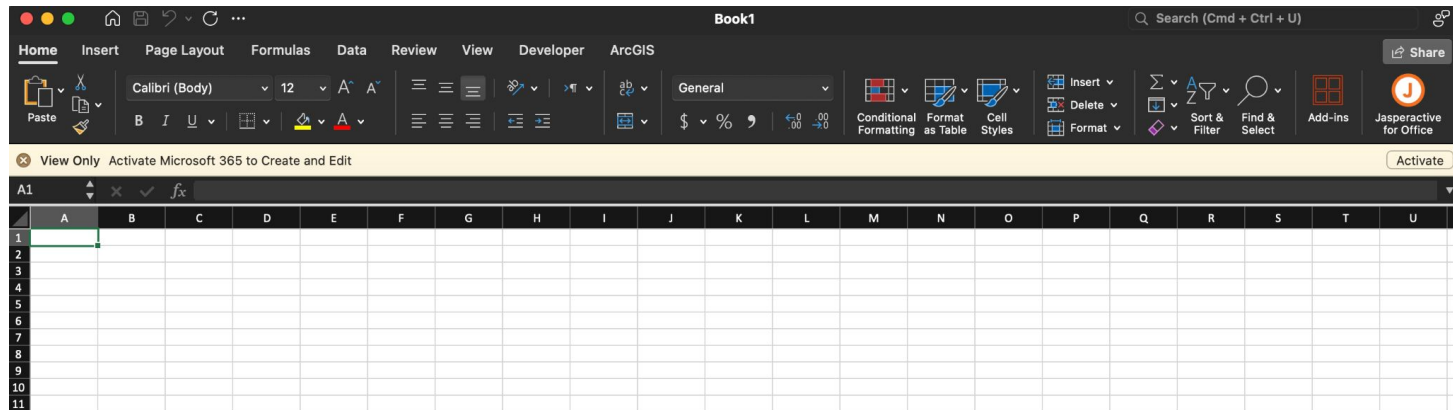
Chapter 1

Importing Data

Let's discuss dataframes

- Dataframes or df for short, are a data structure (recall Python 101)
- They consist of rows and columns
- I'm 100% you've already seen a dataframe

Seem familiar?



Libraries

- We're firstly going to need pandas, the package we will be using extensively
- We can then use any of the read methods (depending on the file type) in pandas to import the dataset we want

```
import pandas as pd
```

```
df = pd.read_csv(filename.csv)
```

read_csv() + read_excel()

- In case you are working with an excel file, you can use the read_excel() method
- There are a lot of optional parameters in these methods
 - I'll teach you one that comes in handy called skiprows
 - It helps you skip rows at the start of the file

```
df = pd.read_csv(filename.csv, skiprows = 1)
```

Exercise time!

We are going to be using Jupyter Notebook for our coding needs today. It's an open-source web-application for writing computational documents. Let's get started!

- Open jupyter.org/try
- Click on JupyterLab
- Let's create a new notebook
 - Click on File -> New -> Notebook
 - Choose the Pyodide Kernel
 - We're good to go!

Exercise 1.1

Take the TikkaMasalaClean.csv file and import it. Do you need to remove any rows?



Indexing

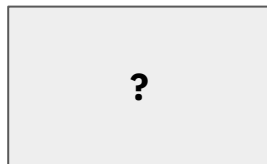
- Remember that indexes in Python start from 0 (much like other programming languages)
- We use indexing to refer to certain parts of a data structure
- Think of it as the page numbers for a data structure

Meat = "Turkey"

T u r k e y

[0] [1] [2] [3] [4] [5]

Meat[1]



u

Slicing

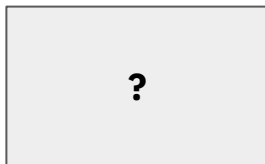
- To find a range of items, we use:
 - “.”
 - e.g `Meat[1:3]` will give you a range of the `Meat` variable including 1 but before 3
 - You can use it in a number of ways:
 - `Meat[1:]` will give you everything after 1
 - `Meat[:3]` will give you everything before 3
 - What will `Meat[:-1]` give you?

`Meat = "Turkey"`

T u r k e y

[0] [1] [2] [3] [4] [5]

`Meat[1:4]`



urk

Navigating a df

- We can access certain parts of a df using a combination of indexing and slicing. Let's look at a few:
 - df.i**loc** uses integers
 - df.l**oc** uses labels
 - In both of these “,” separates rows and columns
- The same logic of indexes and slicing on the previous slide applies

```
df.iloc[row, column]
```

```
df.iloc[row:row, column:column]
```

```
df.loc["row name", "column name"]
```

```
df.loc["row name": "row name", "column  
name": "column name"]
```

Subsetting

- You can filter out specific rows from a df with certain conditions
 - Recall conditional and logical operators from Python 101!

```
df[df["Quantity"] > 4]
```

Example

We learn best when try all of this out. Using the `TikkaMasalaClean.csv` file we imported, let's try all of this out.



C H I L L



Let's take 5 min

Chapter 2

Cleaning and Transforming

What is *dirty* data?

As I see it, data can be nasty in two main ways:

1. Problems with the structure of the data
 - a. We'll be using a paper for this part!
2. Problems within the data

We'll try to cover both as much as we can



Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II.

<http://www.jstatsoft.org/>

Tidy Data

Hadley Wickham
RStudio

Abstract

A huge amount of effort is spent cleaning data to get it ready for analysis, but there has been little research on how to make data cleaning as easy and effective as possible. This paper tackles a small, but important, component of data cleaning: data tidying. Tidy datasets are easy to manipulate, model and visualise, and have a specific structure: each variable is a column, each observation is a row, and each type of observational unit is a table. This framework makes it easy to tidy messy datasets because only a small set of tools are needed to deal with a wide range of un-tidy datasets. This structure also makes it easier to develop tidy tools for data analysis, tools that both input and output tidy datasets. The advantages of a consistent data structure and matching tools are demonstrated with a case study free from mundane data manipulation chores.

Keywords: data cleaning, data tidying, relational databases, R.

[Link to the paper](#)

Tidy Data Principles

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

Hadley
Wickham



CHAPTER 2

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272015272
China	2000	216766	128042583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272015272
China	2000	216766	128042583

observations

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272015272
China	2000	216766	128042583

values

Human readable  **machine-readable**

pd.melt

- This function is essential for when columns are measures instead of variables
- It pivots the data and makes it longer (see right)
- Parameters:
 - **frame** = the dataframe to apply the melt on
 - **id_vars** = columns to pivot on i.e. that remain unchanged
 - **var_name** = name of the variable that is formed from the columns
 - **value_name** = name of the variable that formed from the contents of the columns being melted

country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

table4

[Source](#)

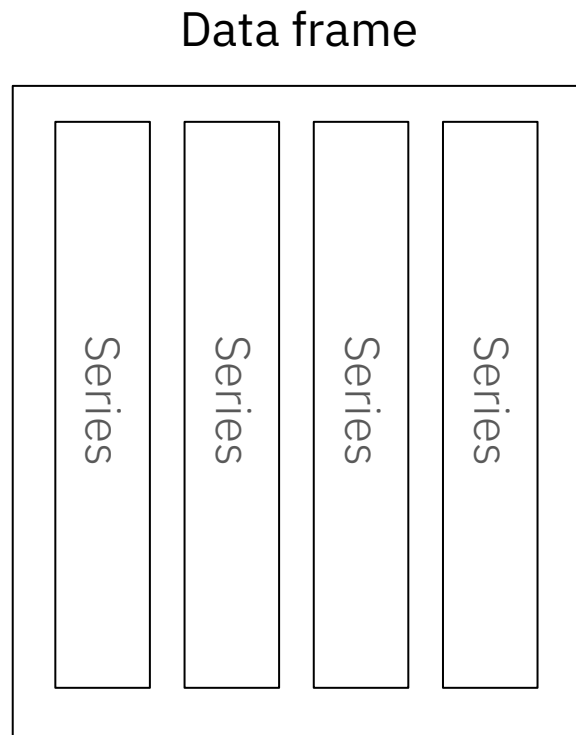
```
pd.melt(frame = df, id_vars = ["country"], var_name =
        "year", value_name = "cases")
```

Example

I have a dataset called `pew-raw`. Let's see how we can tidy it up.

How Pandas sees a df

- A data frame is basically lots of Series stacked together
- And a Series is simply a column in a dataframe
- Combine lots of series together and you get a df
- Series have lots of functions that help data analysts in working with data



df[“column”].str[]

- a function called `str[]` is great for parsing text
 - The numbers you add in the `[]` act the same way as indexing and slicing
- This helpful if you want to split string data
- The code on the right will remove all the characters before the 3rd index in the Ingredient column

```
df[“Ingredient”].str[3:]
```

df.replace()

- This function is a handy tool in cleaning data!
- It finds particular values in a df with the parameter `to_replace`
- And replaces it with what we have in `value`
- For in a particular column, simply add the name of the column (example on the right)
- This is the CTRL + F equivalent for data frames

```
df.replace(to_replace = "Tomatoes",  
          value = "Roma Tomatoes")
```

Or for a particular column

```
df["column_name"].replace("old_value", "new_value")
```

Exercise 2.1

Download `tb-raw.csv`. Can you make the dataset tidy? Can you create a gender column?



BACK TO WORK!

More cleaning!

df.drop()

- This function helps in dropping rows and columns
- When dropping rows, simply add the indexes you want to drop
- When dropping columns, one way is to use the parameter `columns` and add the names of the columns you want to drop

For dropping rows



```
df.drop([0,1,2,3])
```

```
df.drop(columns=['Blah1', 'Blah2'])
```

df.dropna()

- This function helps in dropping rows with missing values
- Using as is will drop NA values from all columns
- Specifying a list of column names within it as **subset** indicates which columns to apply it on

```
df.dropna()
```

```
df.dropna(subset = ["Col1", "Col2"])
```

df.rename()

- This function helps in renaming columns or indexes
- To rename columns you have to add **columns** with current names before a **:** and new names after

```
df.rename(columns={"A": "a", "B": "c"})
```

df.drop_duplicates()

- This function drops any duplicate rows that have the same values across all columns
- Again, you can specify to drop only rows from a particular column by add a **subset** parameter, example on the right

```
df.drop_duplicates(subset=["Col1",  
                           "Col2"])
```

df[“column”].str.strip()

- This function is used to remove whitespace or a particular set of characters

```
df[“Address”].str.strip("/.")
```

Exercise 2.2

Time for you to try it out. Using the `TikkaMasalaDirty.csv` file, do the following:

1. Drop the first row while importing
2. Fix the name of “Ingredieents” column
3. Remove .!, and 12 from the Ingredients column
4. Drop duplicate rows in the “Value” and “Ingredients” column
5. In the “Change Color?” column:
 - a. Replace y with Yes
6. Drop the “Useless” column



BONUS

Homework

Let's explore the Customer Call List dataset

This is going to be your homework. I will send a handout after the workshop.



[Source](#)

Chapter 3

Saving your datasets

Save that clean dataset

- Once we have cleaned up the dataset, we want to store it away
- Our goal is to simply save the work
- Data Engineers (someone working a bit up the pipeline) can follow a medallion architecture on the right
- Raw is the data straight from the source
- Cleaned will work on Raw to fix data quality issues
- Consumption is cleaned with more changes for getting it ready for analytics

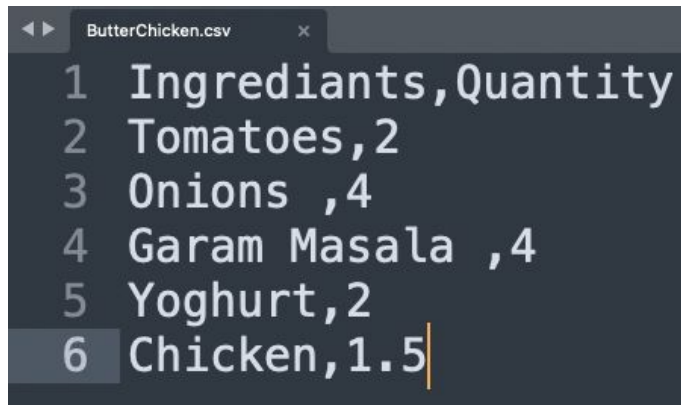


Here's how to do it

- It's straightforward:

```
df.to_csv(filename.csv)
```

- There are [other](#) optional parameters if you need them.
 - e.g. `index = True` will add a column with indexes when you save the file



```
ButterChicken.csv
1 Ingredients,Quantity
2 Tomatoes,2
3 Onions ,4
4 Garam Masala ,4
5 Yoghurt,2
6 Chicken,1.5
```

This is what a comma separated value (csv) file looks like

Exercise 3.1

Save the TikkaMasala dataset we worked in the previous exercises.



Thank you for tuning in!

I am always available for a coffee chat. Drop me a message on LinkedIn (Scan the QR code).

I would love to hear some feedback on the workshop. Please fill out this feedback survey from the iSchool.

I'll see you in Python 103!

